

OGC2026_문제설명(Kor)

최적화 그랜드 챌린지 2026 (OGC 2026)

위대한 조선소 퍼즐: 블록을 채우고, 시간을 이겨라 (The Grand Shipyard Puzzle: Pack the Block, Beat the Clock) OGC 2026 조직위원회 2026년 5월 16일 (문서 버전 1.0)

요약 (Abstract)

최적화 그랜드 챌린지(OGC) 2026은 조선업에서 도출된 공간적 블록 스케줄링(spatial block scheduling) 문제를 다룹니다. 조선소에서 배의 각 구조적 블록은 베이(bay)라고 불리는 고정된 작업 공간 안에서 만들어집니다. 블록은 불규칙한 층상(layered) 형태를 가진 3차원 물체이므로, 이를 겹치지 않게 베이에 배치하는 것 자체가 어려운 기하학적 문제입니다. 이에 더해, 모든 블록에는 릴리즈 날짜(반입 가능일), 처리 시간, 그리고 납기일이 정해져 있으며, 베이 공간은 공유되기 때문에 주변의 다른 블록들로 인해 대기해야 하거나 필요한 시간보다 더 오래 머물러야 할 수도 있습니다. 따라서 알고리즘은 각 블록에 대해 어느 베이로 갈지, 어디에 어떻게 놓일지, 그리고 언제 반입되고 반출될지를 한꺼번에 결정해야 합니다. 이 문서는 공식적인 문제 정의, 참가 가이드, 그리고 평가 기준을 제공합니다.

- 문서 업데이트: 오류를 수정하거나 모호함을 명확히 하기 위해 이 문서는 개정될 수 있습니다. 최신 버전은 항상 대회 웹사이트에서 확인할 수 있으며, 타이틀 페이지의 버전 번호는 현재 읽고 있는 개정판을 나타냅니다. 참가자는 업데이트 사항을 확인하기 위해 웹사이트를 확인할 책임이 있으며, 중요한 변경 사항 역시 웹사이트에 공지됩니다.

1. 문제 설명 (Problem Description)

1.1 용어 (Terminology)

- 베이 (Bay):** 작업이 수행되는 조선소 내의 물리적으로 분할된 작업 공간입니다. 각 베이 $j \in M$ 은 $W_j \times H_j$ 의 고정된 크기를 가지며, $M := 1, 2, \dots, m$ 은 베이의 집합을 의미합니다.
- 블록 (Block):** 배의 대형 구조 부품을 나타내는 생산 단위입니다. 각 블록 $i \in N$ 은 3차원 구조를 형성하는 K_i 개의 다각형 층으로 구성되며, $N := 1, 2, \dots, n$ 은 블록의 집합입니다. 블록은 회전할 수 있으며, 가능한 방향 옵션의 수는 O_i 로 주어집니다.
- 릴리즈 날짜 (R_i):** 블록 i 의 작업이 시작될 수 있는 가장 이른 시간입니다.
- 납기일 (D_i):** 블록 i 의 원하는 완료 시간입니다.
- 처리 시간 (P_i):** 블록 i 를 처리하는 데 필요한 시간입니다.
- 반입 시간 ($ENTRY_i$):** 블록 i 가 베이에 배치되는 시간입니다.
- 반출 시간 ($EXIT_i$):** 블록 i 가 베이를 떠나는 시간입니다.
- 지각 (Tardiness, T):** 납기일을 초과한 지연 시간으로, 다음과 같이 정의됩니다:
$$T_i = \max(0, EXIT_i - D_i).$$
- 작업량 (L_i):** 블록 i 를 처리하는 동안 필요한 총 작업량입니다.
- 선호도 (S_{ij}):** 블록 i 가 베이 j 에 할당되기를 바라는 선호도 점수입니다.

1.2 문제 설정 (Problem Setting)

조선소에서 블록은 여러 생산 단계를 거치며, 그 중 일부는 베이 내에서 수행됩니다. 각 베이는 크레인, 노동력, 운송 경로와 같은 자원이 공유되는 고정된 작업 공간입니다. 블록은 베이에 할당되고 시간적 및 공간적 제약을 모두 만족하면서 처리되어야 합니다. 각 베이는 공간적으로 겹치지만 얹는다면 동시에 여러 블록을 수용할 수 있습니다. 각 블록은 고유한 릴리즈 날짜, 처리 시간, 납기일을 갖습니다. 따라서 할당 및 스케줄링 결정은 공간적 가능성과 시간적 가능성을 동시에 고려해야 합니다.

1.3 베이 (Bays)

각 베이는 독립적인 작업 공간이며, 베이 사이에는 물리적 또는 운영상 분리가 존재합니다. 전형적인 조선소는 여러 베이로 구성되며, 각 베이 j 는 $W_j \times H_j$ 의 크기를 갖습니다. 주어진 블록에 대해 다음 사항들을 동시에 결정해야 합니다:

1. 어느 베이에 할당할 것인가
2. 베이 내 위치 및 방향
3. 언제 블록을 베이에 넣을 것인가
4. 언제 블록을 베이에서 뺄 것인가

우리는 각 베이에 독립적인 크레인이 있다고 가정하므로, 베이 내의 모든 작업은 개별적으로 수행될 수 있습니다. 올해 대회와 관련된 작업은 ENTRY(반입)와 EXIT(반출) 두 가지이며, 각각 블록을 베이에 넣고 빼는 것을 의미합니다. 어느 특정한 날에 각 베이의 크레인 기사는 그날 수행할 작업 목록을 받게 됩니다. 단일 날짜에 수행할 수 있는 작업 수에는 제한이 없으므로, 그날의 본 작업 이전에 임의의 수의 작업이 수행될 수 있습니다. 예를 들어, 처리 시간이 3이고 5일 차에 베이에 배치(ENTRY 작업)되도록 예정된 블록을 생각해보겠습니다. 그렇다면 이 블록이 EXIT 작업을 할 수 있는 가장 이른 날짜는 8일 차입니다. 즉, 이 블록은 5일 차부터 7일 차까지 처리될 수 있습니다. 이 경우 각 작업에 따라 베이의 형태가 달라지므로 작업의 순서도 중요합니다. 그럼에도 불구하고 모든 EXIT 작업은 ENTRY 작업 이전에 완료되어야 한다고 가정합니다. 같은 유형의 작업들 사이에서는 알고리즘이 수행 순서를 결정해야 합니다.

1.4 블록 (Blocks)

총 n 개의 블록이 주어집니다. 각 블록은 3차원 구조를 형성하는 여러 다각형 층으로 구성된다고 가정합니다. 각 층을 정의하는 다각형이 반드시 볼록(convex)한 것은 아닙니다. 예를 들어 직사각형 형태의 층 2개로 구성된 블록은 상단, 전면, 우측 측면도에서 볼 수 있듯 평면 모양이 다릅니다. 각 블록 i 는 K_i 개의 층으로 구성되며 모든 층의 높이는 동일하다고 가정합니다. "비어 있는" 층은 있을 수 없습니다. 또한, 인접한 두 층은 물리적으로 붙어 있는 것이 보장되므로 "떠 있는(floating)" 층은 불가능합니다.

블록이 배치될 때, 블록의 방향도 결정되어야 합니다. 이번 대회에서는 블록마다 가능한 방향 옵션 수가 미리 정해진 문제 인스턴스를 제공하며, 이는 블록 i 에 대해 O_i 로 표기됩니다. 예를 들어 어떤 블록은 8개의 방향 옵션을 가질 수 있습니다. 각 블록 i 는 작업량 L_i 와 베이에 대한 선호도 점수 S_{ij} 를 갖습니다. 작업량은 그 블록에 대해 얼마나 많은 일이 이루어져야 하는지를 나타내며, 베이들 간의 총 작업량 분산이 적은 솔루션이 선호됩니다. 선호도 점수 S_{ij} 는 블록 i 가 베이 j 에 할당되는 것을 얼마나 선호하는지 나타냅니다.

1.5 내려야 할 결정 (Decisions to Make)

각 블록 i 에 대해 다음 사항을 결정해야 합니다:

- 베이 j 로의 할당

- 베이 내 배치 위치
- 방향 선택
- 반입 시간(ENTRY) 및 반출 시간(EXIT)

1.6 제약 조건 (Constraints)

크게 세 가지 제약 조건 그룹이 있습니다. 첫 번째는 모든 블록이 ENTRY/EXIT 작업과 함께 베이에 할당되는 것에 관한 제약입니다. 두 번째는 블록이 최소한 요구되는 처리 시간 동안 베이에 있는 후에만 작업이 수행되도록 하는 시간적 제약입니다. 세 번째는 베이 내 블록의 공간적 가능성에 관한 제약입니다. 크레인 작업이 그날의 본 작업 시간 이전에 수행될 수 있으므로, $ENTRY_i$ 와 $EXIT_i$ 를 가진 블록 i 는 $[ENTRY_i, EXIT_i)$ 동안 할당된 베이를 차지합니다.

1.6.1 할당 제약 (Assignment Constraints)

- **할당 제약:** 모든 블록은 단 하나의 베이에 할당되어야 합니다. 할당되지 않은 블록은 있을 수 없습니다.
- **작업 제약:** 모든 블록은 정확히 한 번의 ENTRY 작업과 한 번의 EXIT 작업을 가져야 합니다. 블록이 베이에 배치되면 이동하거나 회전할 수 없습니다.

1.6.2 시간 제약 (Temporal Constraints)

- **릴리즈 날짜 제약:** 블록은 릴리즈 날짜 이전에 시작될 수 없습니다: $ENTRY_i \geq R_i$.
- **처리 시간 제약:** 블록의 작업은 반출 전에 완료되어야 합니다: $EXIT_i - ENTRY_i \geq P_i$. 블록은 요구되는 처리 시간이 경과한 후에도 베이에 "남아" 있을 수 있습니다. 이는 다른 블록들이 빠져나가는 것을 막을 때 발생할 수 있습니다. 반출 시간 $EXIT_i$ 가 납기일 D_i 를 초과하면 양수의 지각(tardiness)이 발생합니다. 일반적으로 블록은 $D_i - R_i > P_i$ 를 만족하며, 이는 베이 내 다른 블록을 고려하여 더 나은 ENTRY 및 EXIT 시간을 결정할 수 있는 "여유(slack)" 시간이 있음을 의미합니다.

1.6.3 공간 제약 (Spatial Constraints)

방향 o 로 위치 (x, y) 에 있는 블록 i 의 층 l 의 다각형을 $P_{i,l}^{o,x,y}$ 로 표기합니다. 같은 날 동일한 베이에 위치한 두 블록 i_1 과 i_2 를 고려해 봅시다. $int(P_{i_1,l}^{o_1,x_1,y_1}) \cap int(P_{i_2,l}^{o_2,x_2,y_2}) \neq \emptyset$ 이면 두 블록 i_1 과 i_2 사이에 충돌이 있다고 말합니다 (여기서 $int()$ 는 닫힌 다각형의 내부 점들의 집합). 다시 말해, 임의의 층 l 에 대해 $int(P_{i_1,l}^{o_1,x_1,y_1}) \cap int(P_{i_2,l}^{o_2,x_2,y_2}) = \emptyset$ 이면 두 블록은 충돌이 없습니다. 두 블록이 겹치지 않게 다각형의 가장자리를 공유하는 것은 허용됩니다.

다음과 같은 함수 C 를 정의합니다:

$$\nabla(i_1, l_1, o_1, x_1, y_1, i_2, l_2, o_2, x_2, y_2) := \begin{cases} 0, & \text{if } int(P_{i_1,l_1}^{o_1,x_1,y_1}) \cap int(P_{i_2,l_2}^{o_2,x_2,y_2}) = \emptyset, \\ 1, & \text{otherwise.} \end{cases}$$

임의의 날짜 t 와 베이 $j \in M$ 에 대해 다음 집합을 정의합니다:

$N(t, j) := \{i \in N : ENTRY_i \leq t < EXIT_i \text{ and } i \text{ is assigned to } j\}$. 즉, 집합 $N(t, j)$ 는 시간 t 에 동일한 베이 j 에 할당되어 베이에 머무르는 블록들의 부분 집합입니다.

- **베이 포함 제약:** 각 블록은 할당된 베이 내에 완전히 포함되어야 합니다. 예시로 이 제약을 위반하는 블록들이 존재할 수 있습니다.

- **충돌 방지 제약:** 임의의 날짜 t , 베이 j , 그리고 두 개의 서로 다른 블록 $i_1, i_2 \in N(t, j)$ 에 대해, 모든 $l = 1, 2, \dots, \min K_{i_1}, K_{i_2}$ 에서 공간적 충돌 검사 함수가 0이 되어야 합니다. 쉽게 말해, 같은 베이 내 같은 날짜에 있는 두 블록은 같은 높이의 층에서 충돌하지 않아야만 공간적으로 배치가 가능합니다. 예를 들어 위에서 보았을 때는 겹쳐 보여도 층별로는 충돌이 없을 수 있습니다. 반면 층끼리 충돌하는 경우도 있습니다.
- **크레인 작업 제약:** 베이에 블록을 넣거나 빼려면 크레인이 간섭 없이 수직으로 블록을 이동할 수 있어야 합니다. 블록의 크레인 작업(ENTRY 또는 EXIT)은 다른 블록들의 같거나 더 높은 층과 공간적으로 충돌이 없어야만 가능합니다. 특정 블록이 이미 존재하는 다른 블록 위로 걸쳐진 형태라면 크레인 작업 제약을 위반하게 됩니다. 반대로 위에 놓인 블록은 아래 블록이 있어도 크레인 작업이 가능합니다.

1.7 목적 함수 (Objective Function)

목표는 다음 세 가지 기준의 가중치 조합을 최소화하는 것입니다:

- **총 지각 (Total tardiness):** 블록들의 총 지각 합계. $Z_1 := \sum_{i=1}^n T_i$,
- **작업량 불균형 (Workload imbalance):** 베이 간 정규화된 최대 작업량 불균형.
 $Z_2 := [\max_{j_1, j_2 \in M: j_1 \neq j_2} |u_{j_1} \sum_{i \in N(j_1)} L_i - u_{j_2} \sum_{i \in N(j_2)} L_i|]$ 여기서 $N(j)$ 는 베이 j 에 할당된 블록들의 집합이며, u_j 는 각 블록이 놓이는 베이에 부여되는 가중치입니다. 이 값은 전체 베이의 평균 면적을 특정 베이의 면적으로 나눈 값으로 정의됩니다: $u_j = \frac{\sum_{k \in M} W_k \times H_k}{W_j \times H_j}$ 더 큰 베이는 더 작은 가중치 값을 가지며, 이는 동일한 작업량을 가졌다 하더라도 블록을 더 큰 베이에 배치하면 혼잡도가 감소함을 반영합니다.
- **총 선호도 점수 (Total preference score):** 할당된 베이에 대한 선호도 점수 총합.
 $Z_3 := \sum_{j \in M} \sum_{i \in N(j)} (S_i^{max} - S_{ij})$, 여기서 $S_i^{max} := \max_{j \in M} \{S_{ij}\}$ 입니다. 모든 블록이 가장 높은 선호도를 가진 베이에 할당되면 $Z_3 = 0$ 이 됩니다.

따라서 최소화하려는 최종 함수는 다음과 같습니다: $W_1 Z_1 + W_2 Z_2 + W_3 Z_3$ 여기서 w_1, w_2, w_3 는 주어지는 음수가 아닌 가중치 매개변수입니다.

2. 문제 인스턴스 정의 (Problem Instance Definition)

참가자들에게는 OGC 위원회가 생성한 "훈련용(training)" 문제 인스턴스가 제공됩니다. 이 인스턴스들은 실제 조선소 문제를 밀접하게 모방하도록 설계되었으며 다양한 어려운 측면을 포함하고 있습니다. 예를 들어, 어떤 인스턴스는 블록 수가 매우 많고, 다른 인스턴스는 블록 릴리즈/납기일 일정일이 더 빽빽할 수 있습니다. 참가자들은 이러한 다양한 도전 과제를 고려하여 넓은 범위의 상황에서 성능이 좋은 알고리즘을 개발해야 합니다. 단일 문제 인스턴스는 다음과 같이 JSON 형식으로 주어집니다:

- 예제 문제 인스턴스(JSON)의 파일은 네 부분으로 구성됩니다: (1) 문제 이름("name"), (2) 베이 목록("bays"), (3) 블록 목록("blocks"), (4) 목적 함수 가중치("weights").
- 베이의 인덱스는 0부터 $m - 1$ 까지 가정합니다 (m 은 베이 수). 각 베이 $0 \leq j \leq m - 1$ 은 예제에 나온 것처럼 크기가 다를 수 있습니다. 예를 들어 베이 0의 크기는 125×15 입니다.
- 마찬가지로 블록의 인덱스는 0부터 $n - 1$ 까지입니다. 예를 들어 블록 0의 릴리즈 날짜는 37, 납기일은 44, 베이 선호도는 19, 26, 55입니다 ($S_{00} = 19, S_{01} = 26, S_{02} = 55$). 단일 블록에 대한 선호도 점수의 합은 항상 100입니다. 시간에 관련된 모든 값과 선호도 점수는 항상 정수입니다.
- 블록의 "모양(shape)"은 각 방향에 대한 층의 다각형을 설명합니다. 각 방향은 방향 ID("orientation")와 다각형 층 목록("layers")을 가지는 사전(dictionary) 형태입니다. 방향 ID는

항상 정수이며, 다각형 층 목록은 가장 낮은 층부터 가장 높은 층 순서로 정렬된 꼭지점(vertex) 쌍을 포함합니다.

- 예컨대, 블록 0의 방향 ID 0은 두 개의 층으로 구성됩니다. 첫 번째 층(layer 0)의 첫 번째 꼭지점은 항상 $[0.0, 0.0]$ 입니다. 이 꼭지점을 참조점(reference point)이라고 부르며, 동일한 방향 형태의 다른 모든 꼭지점은 이 참조점을 기준으로 한 상대 좌표입니다.
- 알고리즘이 블록 위치를 (x, y) 로 결정하면, 할당된 베이 내에서 해당 블록 모양의 꼭지점 위치들은 베이의 원점으로부터 (x, y) 만큼 평행 이동합니다. 블록의 위치는 층의 다른 점들과 참조점 간의 상대적 위치를 변경하지 않고 참조점을 (x, y) 로 옮기는 방식으로 지정됩니다. 따라서 방향이 다르면 참조점의 상대적 위치가 항상 왼쪽 아래에 있지 않을 수도 있습니다.
- 참조점을 제외한 층의 꼭지점은 최대 4자리의 유효 숫자를 갖는 소수 위치를 가질 수 있습니다. 층의 모양이 꽤 불규칙할 수 있으므로, 블록 간의 공간적 충돌 여부를 체크할 때 수치적 불안정성(numerical instability)이 발생할 수 있습니다. 평가 서버 역시 결과 솔루션의 실현 가능성을 확인할 때 제공된 `utils.py`의 함수를 사용하므로, 알고리즘 최종 단계에서 이 유틸리티 함수를 사용하는 것을 강력히 권장합니다.
- 문제 인스턴스와 알고리즘 솔루션에 포함된 모든 인덱스(베이, 블록 포함)는 0 기반(0-based)입니다. 이는 파이썬(Python)과 더 잘 맞도록 하기 위해 공식 정의와 달리 차이를 둔 것입니다.

3. 알고리즘 제출 및 평가 (Algorithm Submission and Evaluation)

팀들은 알고리즘 코드를 `submission@optichallenge.com` 으로 이메일 제출하여 평가 서버에 올려야 합니다. 알고리즘은 전용 서버에 있는 "숨겨진(hidden)" 문제 인스턴스들을 대상으로 평가되며, 그 결과는 다른 팀들과 순위가 매겨져 대회 웹사이트의 리더보드에 공개됩니다.

3.1 알고리즘 제출 (Algorithm Submission)

서버에 알고리즘 코드를 제출하기 위한 요구 사항은 다음과 같습니다:

- 이메일은 `submission@optichallenge.com` 으로 보내야 합니다.
- 등록할 때 사용한 이메일 주소로 전송해야 합니다.
- 단일 zip 파일을 첨부해야 합니다.
- zip 파일 내에 `myalgorithm.py` 를 포함해야 합니다. 만약 `utils.py` 가 포함된 경우 수정할 수 없습니다 (알고리즘 실행 전에 덮어씌워짐).
- `myalgorithm.py` 파일은 압축 해제된 폴더의 루트(최상위)에 있어야 합니다.
- zip 파일 용량은 15MB를 초과할 수 없습니다.
- 사전 정의된 쿨다운 기간(12시간) 이전에 이전 제출이 승인된 경우 새로운 제출을 할 수 없습니다.

위 규칙을 준수하지 않는 제출은 거부됩니다. 서버는 추가적인 정상 작동 확인(예: 깨진 zip 파일 체크)을 거쳐 유효한 제출을 승인합니다. 승인 후 숨겨진 인스턴스를 풀기 위해 코드가 실행되며, 모든 결과 평가는 가장 최근에 승인된 제출물을 기준으로 합니다. 코드 승인이 알고리즘의 올바른 작동을 보장하는 것은 아니며(오류가 발생하거나 패키지 누락으로 실패할 수 있음) 성공 여부와 무관하게 리더보드에 반영되므로 팀원들은 제출 요건이 모두 충족되었는지 확인해야 합니다. 제출 시스템은 승인/거절 상태를 이메일로 회신하며, 거절 시 간략한 사유를 제공합니다. 거절 사유 수정 후 즉시 재제출이 가능하며 쿨다운 타이머는 승인된 이후부터 작동합니다. 일부 이메일 제공자는 `.dll`, `.vb`, `.exe` 등 첨부파일을 차단할 수 있으므로, 팀들은 첨부 파일이 안전하게 서버로 전송되는지 확인해야 합니다. 제출 기간의 시작과 끝은 웹사이트에 공지되며 횟수 제한은 없으나 남용 시 제한 조치를 받을 수 있습니다.

3.2 알고리즘 평가 서버 (Algorithm Evaluation Server)

팀들은 훈련용 문제를 사용해 알고리즘을 개발할 것으로 기대되며, 제출된 알고리즘은 평가를 위해 서버에서 숨겨진 문제로 실행됩니다. 서버 사양은 다음과 같습니다:

- AMD Ryzen Threadripper PRO 9955WX
- Ubuntu 24.04 LTS

알고리즘은 문제마다 다를 수 있는 시간 제한(Time limit)을 적용받습니다. 이 시간 제한은 대회가 끝난 후에만 공개되지만, 실시간(wall-clock time) 기준으로 몇 분에서 30분 사이일 것으로 예상됩니다. 서버 사양과 참가자의 환경이 다르기 때문에 알고리즘 내부 경과 시간을 꼼꼼히 모니터링하여 한계를 초과하지 않도록 해야 합니다. 시간 내에 답을 내지 못하면 충돌 또는 오작동으로 간주합니다.

공정한 대회를 위해 제출된 알고리즘에는 다음과 같은 제한이 적용됩니다:

- 실행 중 외부 인터넷 접속 금지
- 실행 폴더 상위 디렉터리 접근 금지
- 최대 4개의 CPU 코어(CPU 사용률 400%)
- 최대 16GB의 메모리

제한을 위반하면 실격될 수 있습니다. 이 제한은 firejail 과 cpulimit 을 사용해 적용되므로 알고리즘이 해당 툴과 충돌해서는 안 됩니다. 숨겨진 문제들에 대한 결과는 매시간 집계되며, 집계 시점에 모든 문제의 결과를 도출한 팀들로 리더보드가 생성되어 게시됩니다. 평가는 선입선출 대기열로 처리되므로 제출 시간과 처리 완료 시간은 다를 수 있습니다. 처리가 끝나면 회신 이메일이 발송됩니다.

3.3 리더보드 (Leaderboard)

리더보드 순위는 점수 기반입니다. 각 숨겨진 테스트 인스턴스에 대해 알고리즘이 산출한 결과에 따라 점수가 부여됩니다:

- 솔루션이 불가능(infeasible)하거나, 시간 초과, 또는 충돌한 경우: -1
- 실현 가능한 경우: $R - nb$ (여기서 R 은 평가 대상 팀 수, nb 는 본인 팀보다 객관적으로 더 나은 점수를 낸 팀의 수)

리더보드는 리더보드가 구축될 시점에 모든 문제에서 획득한 총점에 의해 순위가 결정됩니다. 솔루션의 가능성 여부는 `utils.py` 의 `check_feasibility()` 함수로 검증되며, 가능성이 입증되면 시스템이 목적 함수 값을 계산합니다. 최종 우승자는 리더보드 순위뿐만 아니라 기술 보고서 및 발표(프레젠테이션) 등을 종합하여 결정됩니다. 위원회는 최종 평가 가중치를 조정할 권리가 있으며, 단순히 알고리즘 성능뿐만 아니라 새롭고 이론적인 학문적 접근도 높게 평가합니다.

4. 알고리즘 개발 및 베이스라인 (Algorithm Development and Baseline)

4.1 개발 환경 (Development Environment)

각 팀은 자체 로컬 머신에서 알고리즘을 테스트해야 합니다. 제출될 알고리즘은 miniforge 를 사용해 만든 Conda 환경에서 실행되며, 이 환경 파일은 웹사이트에서 받을 수 있습니다. (환경 파일 예시 버전: Python 3.12, pandas 2.2.3, scipy 1.15.2, scikit-learn 1.6.1, pytorch 2.11.0 등) 팀들은 필요한 다른 패키지를 zip 파일에 함께 묶어서 제출할 수도 있습니다. 하드웨어나 OS에 따라 환경 생성이 실패할 수 있으며(특히 텐서플로 등), 문제가 되는 패키지를 쓰지 않으면 yml 파일에서 삭제하고 재시도하면 됩니다. 이 환경은 단순 서버 호환성을 높이기 위함이며 의무사항은 아닙니다. 추가 패키지 설

치를 요청할 수 있으며, 위원회가 설치 여부를 결정합니다. 질문이나 요청을 위해 디스코드 서버가 운영될 예정입니다. 평가 서버에는 전용 GPU가 없으므로 머신러닝 알고리즘은 CPU에서만 돌아가도록 해야 합니다. Gurobi와 FICO의 지원으로, 제출물에 상용 MIP 솔버(Gurobi 13.0.2, Xpress 9.8.1)를 사용할 수 있습니다.

4.2 베이스라인 알고리즘 (Baseline Algorithm)

웹페이지에는 탐욕(greedy) 알고리즘 방식의 베이스라인이 제공됩니다. `myalgorithm.py` 파일이 알고리즘의 진입점이 되며, 서버는 이 파일 안의 `algorithm(prob_info, timelimit)` 함수를 찾습니다. 이 함수는 `prob_info` 딕셔너리와 `timelimit` 을 인자로 받아 솔루션 정보를 담은 딕셔너리를 반환해야 합니다. 솔루션 딕셔너리는 유일한 키(key)인 "operations" 를 가지며, 이 값은 날짜(시간) 인덱스를 키로 하는 또 다른 일일 작업 딕셔너리입니다. 작업이 있는 날짜 인덱스만 포함하면 됩니다. 각 날짜별로 "ENTRY" 및 "EXIT" 두 종류의 리스트가 나열됩니다. 일일 작업 수에는 제한이 없지만 "EXIT" 이 "ENTRY" 보다 먼저 수행되어야 합니다. 모든 숫자는 정수여야 하며 블록의 위치(x, y)는 분수(소수)를 가질 수 없습니다. 평가 시스템은 실현 가능성을 체크하기 전 위치 값을 항상 반올림(round)합니다. 베이스라인은 납기일이 빠른 순(EDD) 규칙을 따르는 가장 단순한 방식을 구현하고 있습니다. 이때 크레인 제약 조건이 초기에 무시될 수 있으므로 베이스라인 코드는 불가능한 블록들을 베이가 비워지는 나중 날짜로 넘겨버립니다. 팀들은 이를 참조용으로 사용할 수 있습니다.

4.3 알고리즘 테스터 및 시각화 (Algorithm Tester and Solution Visualization)

알고리즘을 쉽게 테스트하고 시각화할 수 있는 툴이 제공됩니다. 대회 웹사이트에서 `alg_tester.zip` 을 받아 압축을 푼 뒤, Conda 환경을 활성화하고 `python alg_tester_app.py` 를 실행하세요. 여기서 문제 인스턴스를 고르고 알고리즘 파일이 있는 폴더를 지정하면, "Problem" 탭과 "Solution" 탭에서 상세 내용을 보고 솔루션을 시각적으로(단면도 등) 확인할 수 있습니다. 이 툴은 유용한 참조용이며, 발견되는 버그는 보고해주시면 감사하겠습니다.

5. 대회 형식 및 정책 (Competition Format and Policy)

대회는 예선(Preliminary)과 결선(Final) 두 단계로 진행되며, 참가자는 최적화, 머신러닝, 휴리스틱 등 어떤 방식이든 자유롭게 사용할 수 있습니다. 리더보드는 정확한 등수가 아닌 티어(Tier)별 그룹 순위로 노출되며, 실제 정확한 순위 평가는 각 스테이지 종료 후에 공개됩니다.

5.1 예선 (Preliminary Stage)

1. **자동 평가:** 참가자가 알고리즘을 제출하면 서버에서 이를 실행해 성능을 평가하고 티어를 업데이트합니다.
2. **결선 진출:** 자동 평가 결과와 조직위의 코드 검증을 기반으로 상위 30~40개 팀(변경 가능)을 결선 진출팀으로 선발하며, 종료 시점에 순위를 공개합니다.

5.2 결선 (Final Stage)

1. **자동 평가:** 예선과 동일하게 알고리즘 성능 점수를 측정합니다.
2. **기술 보고서 제출:** 결선 진출팀은 정해진 양식에 따라 알고리즘을 설명하는 보고서를 제출해야 하며, 코드 퀄리티와 함께 조직위가 별도로 평가합니다.
3. **발표 (Presentation):** 결선 진출팀은 접근 방식과 결과에 대해 조직위 앞에서 프레젠테이션을 진행해야 합니다.

4. **최종 선정:** 점수, 보고서, 코드 평가, 발표 등을 종합적으로 심사하여 최종 수상팀을 선정하고 순위를 최종 공개합니다.

5.3 평가, 공개 및 청렴성 (Evaluation, Disclosure, and Integrity)

구체적인 평가 기준과 절차는 조직위의 재량에 따릅니다. 수상팀 외 개별 순위와 평가 내역은 공개되지 않습니다. 결선 진출팀의 모든 코드와 기술 보고서는 대중에 공개되므로 오픈소스화가 가능한 형태여야 하며, 대회 참가로 이에 동의하는 것으로 봅니다. 결과 발표 이후에도 표절이나 부정행위가 적발되면 수상이 취소될 수 있습니다.

6. 조직위원회 (Organizing Committee)

최적화 그랜드 챌린지 2026은 다음 인원들에 의해 기획 및 운영됩니다:

- 이경식 (서울대학교, 의장)
- 이충목 (한국외국어대학교)
- 우종훈 (서울대학교)
- 강찬목 (숭실대학교)
- 정슬기 (아주대학교)
- 정윤우 (LG CNS)
- 안기백 (LG CNS)
- 이종민 (LG CNS)